

PRIMERA SOLEMNE PROGRAMACIÓN AVANZADA CIT1110 2do Semestre 2026

Instrucciones:

- Lea cada pregunta por completo antes de comenzar a contestar.
- Esta solemne tiene un total de 60 puntos, usted cuenta con 120 minutos para contestar, administre bien su tiempo.
- Recuerde que esta y todas las actividades evaluativas de este curso están sujetas al reglamento de la universidad referente a los actos de plagio, copia y cualquier actividad que implique la no-autoría de su entrega, incluyendo el uso de herramientas de generación automática de código (ChatGPT, Claude u otros).

PARTE I: Verdadero o Falso (9 puntos total; 1,5 c/u)

Justifique brevemente todas sus respuestas relacionadas a programación en Java (independientemente de que si es V o F):

1. La instrucción `byte valor = (byte) 200;` compila sin errores, aunque el valor que queda almacenado en `valor` no es 200.
2. Si `char letra = 'a';`, la instrucción `char siguiente = letra + 1;` compila sin errores y deja almacenado en `siguiente` el carácter 'b'.
3. Si no hay bloques `else`, escribir `if (a) { if (b) { X } }` produce exactamente el mismo comportamiento que escribir `if (a && b) { X }`.
4. La variable de iteración (`i`) declarada en la sección de encabezado de una instrucción `for` puede utilizarse (por ej., usarla en una operación o imprimirse) fuera del ciclo.
5. Las instrucciones `float dato = datos[0]; dato = dato + 1;` modifican el primer elemento del arreglo `datos`, incrementando su valor en 1.
6. Si tenemos la siguiente operación lógica `x && !y`, en caso `x` sea `False`, el compilador no requiere verificar el valor de `!y` para determinar el valor de verdad final de la expresión.

PARTE II: Traza y Descripción (11 puntos)

Para el siguiente algoritmo: rellene la tabla de ejecución adjunta con la cantidad de filas necesarias equivalente a la cantidad de iteraciones del programa. Luego, escriba la salida en pantalla esperada y describa el propósito general del programa (*qué es lo que hace*) en una o dos frases usando sus propias palabras.

Java

```
// fila = estudiante, columna = clase; 1 = asistio, 0 = no asistio
int[][] asistencia = {
    {1, 0, 1, 1},
    {1, 1, 1, 0},
    {0, 0, 1, 1}
};
int[] presentes = new int[4];
int clasesCriticas = 0;

for (int c = 0; c < 4; c++) {
    for (int f = 0; f < 3; f++) {
        if (asistencia[f][c] == 1) {
            presentes[c] = presentes[c] + 1;
        }
        /* Checkpoint 1 */
    }
    if (presentes[c] < 2) {
        clasesCriticas = clasesCriticas + 1;
    }
    /* Checkpoint 2 */
}
System.out.println(clasesCriticas);
```

Clase (c)	presentes[c] en checkpoint 1 tras f = 0	presentes[c] en checkpoint 1 tras f = 1	presentes[c] en checkpoint 1 tras f = 2	clasesCriticas al terminar la iteración (checkpoint 2)

PARTE III: Implementación

Por cada problema, implemente en lenguaje Java lo que se solicita, incluyendo la clase y método main, así como los *imports* necesarios (en caso de necesitarlos).

Problema 1 (20 puntos):

Un almacén desea automatizar el cobro mediante una caja de autoatención. El sistema cuenta con dos arreglos paralelos de 357 elementos con la información de los productos:

Código inicial

```
String[] articulos = new String[357]; // Almacena los nombres
int[] precios = new int[357];          // Almacena el precio unitario de
cada producto
```

(Copie estas líneas a su programa y asuma que contienen los datos necesarios)

Su programa debe seguir este orden estricto:

- Muestre un mensaje de bienvenida al almacén.
- Solicite e ingrese cuántos tipos de artículos distintos llevará el cliente.
- Para cada tipo de artículo solicitado:
 - Pida el nombre del artículo (ingreso por teclado).
 - Pida la cantidad de unidades de dicho artículo que desea llevar (≥ 1).
 - Busque el nombre del artículo en el arreglo artículos (asuma para este ejercicio que el nombre ingresado siempre existe en el arreglo y es único).
 - Calcule el subtotal (precio * cantidad) y acumúlelo al total de la compra. Asuma que todos los artículos tienen stock ilimitado.
- Al finalizar de procesar todos los artículos, imprima en pantalla el monto total a pagar.

Problema 2 (20 puntos):

Escriba un programa en Java que trabaje con una matriz de tamaño $N \times M$, cuyos valores se encuentran definidos previamente. Cada posición de esta matriz representa un píxel de una imagen en escala de grises, por lo que sus valores estarán comprendidos entre 0 y 255 (ver imagen 1). Ver *Código inicial* al final de este enunciado: por simplificación se define una matriz de 3x3 pero su solución debe funcionar para cualquier $N \times M$.

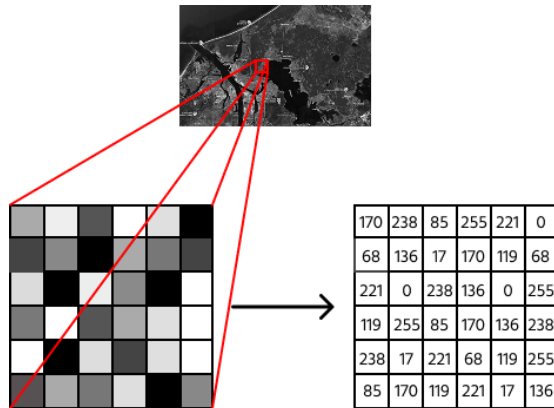


Imagen 1. Representación matricial de imagen como valores en escala de grises

Su programa debe implementar un filtro de representación matricial de imágenes que haga lo siguiente:

- Calcular el promedio de todos los valores almacenados en la matriz m .
- Solicitar al usuario un valor de reemplazo, que debe estar entre 0 y 255, inclusive.
 - Si el usuario ingresa un valor fuera de este rango, deberá solicitarlo nuevamente hasta que sea válido.
- Recorrer la matriz m y reemplazar, utilizando el valor ingresado por el usuario, todos aquellos valores que se encuentren bajo el promedio de la matriz. Su solución debe funcionar para cualquier matriz m , no sólo para la definida al comienzo del código inicial).
- Mostrar la matriz m resultante en pantalla después de realizar todos los reemplazos pertinentes.

Código inicial

```
int[][] m = { {100, 10, 20},  
              {200, 20, 35},  
              {10 , 10, 45} };
```

Ejemplo de ejecución dado el código inicial

```
Ingrese el valor de reemplazo (0-255): 300  
Valor inválido. Ingrese nuevamente: -5  
Valor inválido. Ingrese nuevamente: 80  
Matriz resultante:  
100 80 80  
200 80 80  
80 80 80
```